

Softwares de Sistemas

Java Virtual Machine

SOFTWARE BÁSICO, TURMA A
PROF. MARCELO LADEIRA CIG/UNB

Baseado no livro "The Java Virtual Machine Specification", Second Edition, Tim Lindholm & Frank Yellin. Parte 1 de 3

Sumário

Estruturação da JVM

- Introdução
- Tipos de dados
- Tipos primitivos e valores de dados
- Tipos de referência e valores de dados
- Estruturas de dados em tempo de execução
- Frames
- Representação de objetos

Instruções da JVM

Tipos de Dados

Primitivos

- numérico (integral e ponto flutuante), booleano e o tipo returnAddress
- assume apenas valores primitivos

De referência (ponteiros para objetos)

- assumem apenas valores de referência

Ambos os tipos de valores podem ser

- armazenados em variáveis, passados como argumentos, retornados por métodos e utilizados em operações específicas.

Tipos de Dados

Comentários

- A JVM não realiza checagem de tipo
 - Java é “fortemente tipada” e usa checagem de tipo estática
- Valores de tipos primitivos
 - não possuem tags e nem requerem checagem dinâmica para serem distinguidos de valores de tipos de referência.
- Instruções da JVM
 - distinguem os tipos dos operandos usando instruções projetadas para operar com valores de tipos específicos.
 - iadd, ladd, fadd e dadd (..., value1, value2 ⇒ ..., result)
 - somam 2 valores numéricos e produz um resultado numérico.
 - cada qual é específica para um tipo de operando: int, long, float, e double, respectivamente.

Tipos de Dados

Comentários

- Um objeto é uma instância de uma classe ou um array.
 - referência para um objeto é um dado do tipo de referência.
 - valores de tipo de referência podem ser vistos como ponteiros para objetos
 - mais de uma referência pode existir para um objeto
- objetos são sempre operados, passados ou testados via valores de tipos de referência
 - não existem ponteiros em Java (na forma similar na linguagem C).

Se existir mais de uma referência para um objeto, como o coletor de lixo sabe quando desalocá-lo?

Tipos Primitivos e Valores de Dados

Tipos integrais

- byte $(-2^7 \text{ to } 2^7-1)$ -128 a 127
- short $(-2^{15} \text{ to } 2^{15}-1)$ -32768 a 32767
- int $(-2^{31} \text{ to } 2^{31}-1)$ -2147483648 a 2147483647
- long $(-2^{63} \text{ to } 2^{63}-1)$ -9223372036854775808 a 9223372036854775807
- char $(0 \text{ to } 2^{16}-1)$ 0 a 65535

Tipos ponto flutuante (IEEE 754)

- float, cujos valores são elementos do conjunto de valores float
- double, cujos valores são elementos do conjunto de valores double

Tipos booleanos

- valores verdade true (1) e false (0)
- Não há instruções JVM específicas para tipo booleano
 - Expressões booleanas em Java são mapeadas como expressões **int** na pilha de operandos
 - Arrays do tipo booleano são mapeados em **bytes** na JVM (8 bits)

Tipos Primitivos e Valores de Dados

Tipo returnAddress

- único tipo não associado com um tipo de dados da linguagem Java.
 - usado pelas instruções **jsr** (jump subroutine), **ret** (return from subroutine) e **jsr_w** (jump subroutine – wide index) da JVM
- seus valores são ponteiros para código de operação indicando desvio incondicional no **método** sendo executado
 - **jsr** branchbyte1 branchbyte2 ($\dots \Rightarrow \dots, \text{address}$)
 - **jsr_w** branchbyte1 branchbyte2 branchbyte3 branchbyte4 ($\dots \Rightarrow \dots, \text{address}$)
 - **ret** index ($\dots \Rightarrow \dots$)
 - index é um offset de um byte utilizado como um índice para o vetor de variável locais do frame que contém o offset de retorno. A execução continua nesse offset.

offset de retorno (tipo returnAddress) é o endereço da instrução seguinte.

Tipos de Referência e Valores de Dados

Tipos de classes

- Valores são referências a instâncias de classes criadas dinamicamente

Tipos de array

- Valores são referências a arrays criados dinamicamente

Tipos de interface

- Valores são referências a instâncias de classes que implementam interfaces ou arrays

Valor de referência especial null

- referência vazia (indica que não aponta um objeto)
- não tem tipo em runtime mas pode ser moldada para qualquer tipo de referência
- a especificação da JVM não define um valor concreto para codificar null

Estruturas de Dados em Tempo de Execução

Alocadas na iniciação da JVM

- Heap
- Área de métodos
 - Pool de constantes

Alocadas na iniciação de threads

- Registro PC
- Pilha da JVM
- Pilha de método nativo

Estrutura de Dados em Tempo de Execução

Registro PC

- Cada thread da JVM possui seu próprio registro PC (contador de programa).
- Em um certo instante, cada thread executa o código de um único método que é denominado **método corrente**
 - **método corrente é não nativo (método Java)**
 - o registro pc contem o endereço da instrução da JVM sendo executada.
 - **método corrente é nativo (método não Java)**
 - o valor do registro pc é indefinido.
 - pc pode conter um returnAddress ou um ponteiro nativo na plataforma especifica em que a JVM foi implementada

Estrutura de Dados em Tempo de Execução

Pilha da JVM

- armazena frames associados a cada chamada de um método em uma thread.
 - Permite manipular variáveis locais, resultados parciais e participa da chamada e retorno de um método.
 - A implementação da JVM deve permitir pilha de tamanho fixo ou expansão e contração dinâmica como requerido pela computação.
 - Pode gerar as seguintes exceções
 - **StackOverflowError** se a computação em uma thread requer uma pilha maior do que a permitida.
 - **OutOfMemoryError** se não tem memória suficiente para expandir dinamicamente uma pilha ou para criar uma pilha para uma nova thread.

O tamanho máximo de uma pilha é definido pela opção não padronizada "-Xss<size>" da JVM (programa java)

Estrutura de Dados em Tempo de Execução

Heap

- O heap é criado na iniciação da JVM e compartilhado por todas as threads da JVM.
 - É utilizado para alocar memória para os objetos (todas as instâncias de classes e arrays)
 - Objetos **não são explicitamente desalocados**
 - Objetos sem caminhos de acessos são coletados por um coletor de lixo.
 - A implementação da JVM deve permitir controlar o seu tamanho inicial, se pode ser dinamicamente expandido ou contraído e seu tamanho máximo e mínimo.
 - Pode gerar a seguinte exceção
 - **OutOfMemoryError** se uma computação requer mais memória do heap do que o que pode ser disponibilizado.

Os valores inicial e máximo do heap são definidos, respectivamente, pelas opções "Xms<size>" e "Xmx<size>" da JVM (programa java)

Estrutura de Dados em Tempo de Execução

Área de Métodos

- Essa área é criada na iniciação da JVM e compartilhada por todas as threads da JVM (caso do computador ser multi-core).
 - similar a área de armazenamento de código compilado de uma linguagem compilada
 - armazena estruturas de dados por **classe**
 - pool de constantes, atributos (fields) e dados de métodos, código de métodos e código de construtores.
 - Pode gerar a seguinte exceção
 - **OutOfMemoryError** se o requisito para a área de memória do método não pôde ser atendido.

Estrutura de Dados em Tempo de Execução

Pool de Constantes (CP)

- Construído na área de método quando a classe/interface for criada pela JVM.
 - armazena estrutura de dados por **classe** ou **interface**
 - representação **runtime da tabela constant_pool** do arquivo .class
 - contém constantes, de literais numéricos conhecidos em tempo de compilação a referências a métodos e atributos que devem ser resolvidos em tempo de execução.
 - desempenha uma função similar a de uma **tabela de símbolos** em linguagem compilada
 - Pode gerar a seguinte exceção
 - **OutOfMemoryError** se a construção do pool de constantes para uma classe ou interface requer mais memória do que pode ser disponibilizada na área de método da JVM.

Estrutura de Dados em Tempo de Execução

Pool de constantes

Comentários

- Armazena todas as constantes associadas com a classe ou interface
 - inclui os valores das variáveis declaradas como **final**
- Evita duplicar as constantes em cada instância da classe.
 - instruções que empilham constantes na pilha de operandos a partir do CP utilizam um **índice de constante de pool** como argumento (operando)
 - de 1 byte (1 a 255) ou 2 bytes (1 a 65535) após o bytecode
 - **ldc index** ($\dots \Rightarrow \dots, value$)
 - index é um índice de **um byte** para o pool de constante
 - empilha um int, float ou referência a uma string literal na pilha de operandos
 - **ldc_w indexbyte1 indexbyte2** ($\dots \Rightarrow \dots, value$)
 - indexbyte é um índice de **dois bytes** para o pool de constante
 - **ldc2_w indexbyte1 indexbyte2** ($\dots \Rightarrow \dots, value$)
 - empilha um long ou double na pilha de operandos

Estrutura de Dados em Tempo de Execução

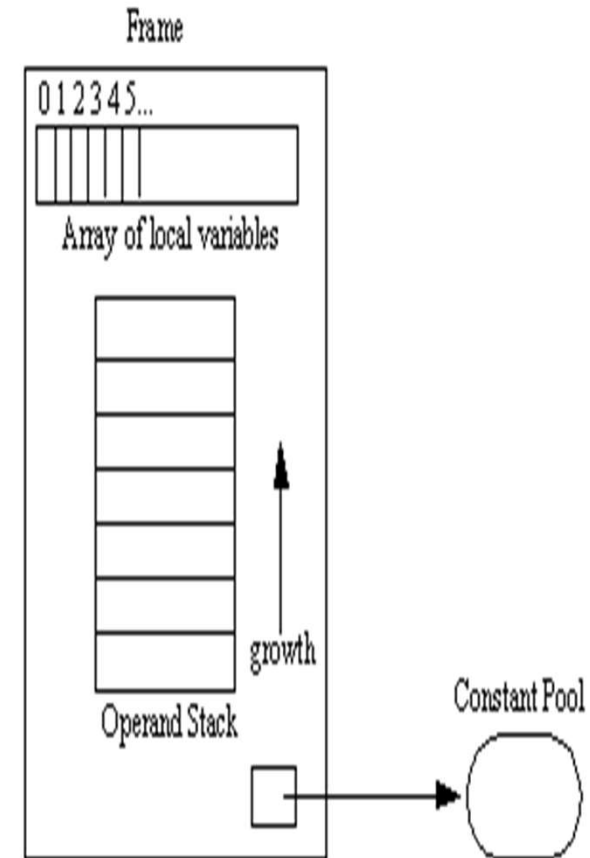
Pilha de Método Nativo

- É utilizada como suporte a métodos **nativos** (não Java).
 - É uma pilha convencional conhecida coloquialmente como **pilha C**
 - É alocada tipicamente por thread quando a thread é criada.
- Pode gerar as seguintes exceções
 - **StackOverflowError** se a computação em uma thread requer uma pilha de método nativo maior do que a permitida.
 - **OutOfMemoryError** se a pilha de método nativo pode ser dinamicamente expandida mas não há memória suficiente ou se não existe memória suficiente para criar a pilha de método nativo para uma nova thread.

Pilha de Frames

Frame é usado para armazenar dados e resultados parciais, executar ligação dinâmica, retornar valores para métodos e disparar exceções.

- Um novo frame é criado cada vez que um método é chamado.
 - O frame é destruído quando o método que o chamou encerra normalmente ou lança exceção.
- Alocado na pilha da JVM da thread que o criou.
 - Cada frame possui seu próprio **array de variáveis locais**, **pilha de operandos** e **referência para o pool de constantes** da classe do método corrente.
 - Slots do array de variáveis locais e da pilha de operandos são de 32 bits.



Pilha de Frames

Comentários

- Os tamanhos do array de variáveis locais e da pilha de operandos são determinados em **tempo de compilação** (maxlocals e maxstack)
 - são armazenados junto com o código do método associado com o frame.
 - a memória para as estruturas de dados do frame pode ser alocada simultaneamente com a chamada do método
- Frame, método e classe correntes
 - frame para o método em execução (**método corrente**)
 - é o único ativo em uma thread (**frame corrente**).
 - a classe que define o método é a **classe corrente**
 - **as operações sobre variáveis locais e pilha de operandos** são referenciadas ao frame corrente.

Pilha de Frames

Comentários

- Um frame deixa de ser o corrente se seu método chamar outro método ou se finalizar.
 - quando um método é chamado, um **novo frame** é criado.
 - esse frame **torna-se o corrente** quando o controle é transferido para o novo método.
 - no retorno do método, o frame corrente **passa o valor resultante da chamada do método**, se existir, para o **frame prévio**.
 - o frame corrente é descartado e o frame prévio torna-se o novo frame corrente.
 - um frame criado por uma thread é local a thread e não pode ser referenciado por qualquer outra thread

Pilha de Frames: Vetor de Variáveis Locais

Vetor (tabela) de variáveis locais

- contêm os **parâmetros do método e valores das variáveis locais** associadas a classe ou interface que gerou o frame
 - comprimento é determinado em **tempo de compilação** e armazenado junto com o código do método associado com o frame
 - Os parâmetros são armazenados primeiro, iniciando no **índice 0**.
 - frame de método construtor ou de instância: referência é armazenada no índice 0 e é seguido pelos parâmetros.
 - método estático: o primeiro parâmetro é armazenado no índice 0, e assim por diante.
- variável local pode armazenar um valor do tipo **boolean**, **byte**, **char**, **short**, **int**, **float**, de **referência** ou **returnAddress**.
- Slots são de 32 bits
 - **longs e doubles** utilizam dois slots (acoplados) em **big-endian order**

Um inteiro é considerado um índice do vetor de variáveis locais `sss` assume um valor entre 0 e o tamanho do array menos 1.

Pilha de Frames: **Pilha de Operandos**

Usada para armazenar valores que são operandos de instruções da JVM.

- tamanho determinado em tempo de compilação.
- a pilha é criada vazia quando o frame é criado
 - algumas instruções empilham valores na pilha.
 - outras retiram operandos da pilha, manipula-os e empilham o resultado.
 - Por exemplo, iadd retira os dois inteiros no topo da pilha, soma-os e empilha o resultado
(..., value1, value2 $\Rightarrow$..., result)
 - valores retornados pelos métodos são recebidos via pilha de operandos.
- Slots são de 32 bits
 - longs e doubles utilizam dois slots (acoplados) em big-endian order

Pilha de Frames: **Ligação Dinâmica**

Referências para o CP permite suportar ligação dinâmica do código do método

- o código no formato .class para um método usa referência simbólica para os métodos que serão chamados e para as variáveis a serem acessadas.
 - **ligação dinâmica** traduz as referências simbólicas em:
 - referências concretas a métodos
 - se necessário, **classes são carregadas para resolver os símbolos ainda não resolvidos**
 - offsets apropriados em estruturas de armazenamento com a localização em tempo de execução das variáveis (loais e dos fields)
 - **ligação tardia** dos métodos e variáveis torna alterações em outras classes que um método usa menos prováveis de quebrar o código desse método.

Pilha de Frames: **Término Normal de Método**

Término normal ocorre se a execução do método não provocar o **lançamento de exceção**

- o método chamado executa uma das instruções **return** com o tipo apropriado para o tipo do valor a retornar (**areturn, dreturn, freturn, ireturn, lreturn**)
 - se nenhum valor for retornar (método com **void**), executa **return** apenas.
- o frame do método chamador torna-se o **corrente** para restaurar o contexto para o método chamador
 - inclui suas variáveis locais, pilha de operandos e o PC é atualizado para apontar para a próxima instrução seguinte a invocação do método chamado.
- execução continua no frame do método chamador com o **valor retornado** (se existente) **empilhado na pilha de operandos** desse frame.

Pilha de Frames: **Término Abrupto de Método**

Ocorre se a execução do método provocar o lançamento de uma exceção que não é manipulada dentro do método

- um método que termina abruptamente não pode retornar um valor para o método chamador
 - O processamento é encerrado

Informação adicional

- Um frame pode ser estendido com informação adicional específica tal como informação de debug

Representação de Objetos

A especificação da Oracle não determina nenhuma estrutura particular interna para representar objetos

- implementação da Sun para a JVM
 - uma referência para uma instância de classe é um ponteiro para um *handler* que é um par de ponteiros:
 - um para uma tabela contendo os métodos do objeto e um ponteiro para a classe que representa o tipo do objeto.
 - O outro ponteiro é para a memória alocada para os dados do objeto

Operações de ponto flutuante e o padrão IEEE Java

Java implementa um subconjunto do padrão 754-1985

- Ponto flutuante não lança exceções, traps ou sinaliza erros
 - operador inválido, divisão por zero, overflow, underflow
 - não sinaliza a ocorrência de NaN
- Modo de ponto flutuante **FP-strict**
 - Usa norma IEEE 754 para float (32 bits) e double (64 bits)
 - garante mesmo resultado, independentemente do processador
- Modo de ponto flutuante **non FP-strict**
 - Usa float-extended-exponent e double-extended-exponent
 - Sem arredondamento, se usar **80 bits**. Valor diferente se usar IEEE 754

Tendência é usar FP-strict para garantir o mesmo resultado, com uso da norma IEEE 754

Palavra chave (não pode ser usada como identificador)

abstract	continue	for	new	switch
assert	default	if	package	synchronized
boolean	do	goto	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

As palavras chaves **const** e **goto** não estão implementadas atualmente.

`assert exprBooleana : <se a mensagem é falsa, encerra o processamento>;`

`true, false e null são literais (constantes), não são palavras chaves`

Métodos especiais

Método iniciação de instância

- com o mesmo nome da classe, o método construtor é mapeado no arquivo .class para **<init>**
 - é executado quando um objeto dessa classe é instanciado
 - não pode ser chamado em um programa Java
 - se houver mais de um construtor, ao executar deve chamar outro
 - são identificados por possuírem assinaturas diferentes
 - não pode retornar um valor
 - não pode ter o modificador **static** ou **strictfp**
 - é chamado via a instrução **invokespecial** durante a execução na JVM

Métodos especiais: Método de Iniciação de Instância

```
class Point {  
    int x, y;  
    Point(int x, int y) { this.x = x; this.y = y; }  
}  
class ColoredPoint extends Point {  
    static final int WHITE = 0, BLACK = 1;  
    int color;  
    ColoredPoint(int x, int y) {  
        this(x, y, WHITE);  
    }  
    ColoredPoint(int x, int y, int color) {  
        super(x, y);  
        this.color = color;  
    }  
}
```

Métodos especiais: Método Iniciação de Classe

o método construtor é mapeado no arquivo .class para `<clinit>`

- utilizado para inicializar **fields static** e **blocos declarados static**
- não pode ser chamado em um programa Java
 - só pode existir um na classe
 - não tem argumentos e não retorna nenhum valor
- Se existir, é o primeiro método a ser executado

Métodos especiais: Método Iniciação de Classe

```
class static_test{  
    static int a = -1;  
    static double b = -2.0;  
  
    public static void main(String argv[]) {  
        System.out.println(a);  
        System.out.println(b);  
        a = 3;  
        System.out.println(a);  
    }  
}
```

Métodos construtores: **ordem de execução**

BLOCO ESTÁTICO

```
class Foo extends Goo {  
    static { System.out.println("1"); }  
    { System.out.println("2"); }  
    public Foo() {  
        System.out.println("3"); }  
    public static void main(String[]  
args) {  
        System.out.println("4");  
        Foo f = new Foo();  
    }  
}
```

EXECUTADO ÚNICA VEZ!

```
class Goo {  
    static {  
        System.out.println("5"); }  
    { System.out.println("6"); }  
    Goo() {  
        System.out.println("7");  
    }  
}
```

- 5 (**bloco static, super classe**)
- 1 (**bloco static da classe**)
- 4 (execução de **main**)
- 6 (**Foo extends Goo, bloco**)
- 7 (Construtor Goo)
- 2 (**bloco de Foo**)
- 3 (construtor de Foo)